

Fatwa Bot Platform — Project Foundation

Read everything inside the `prompts/` folder before doing anything else.

Also inspect the `design/` folder completely.

The designs inside the folder are the **current reference UI**, but they are **not** the final product.

Treat them as the baseline.

You are encouraged to improve the UX, UI, navigation, animations, accessibility, feature set, user flows, architecture, and overall product quality whenever you believe it results in a better application.

Your responsibility is not to reproduce the existing application.

Your responsibility is to build a significantly better product.

Your Role

Act as:

- Principal Software Architect
- Senior iOS Engineer
- Senior Android Engineer
- Senior Backend Engineer
- Senior System Designer
- Senior Product Engineer
- Senior UI/UX Engineer

Think like a company building the best Islamic companion application, not simply another AI chatbot.

Challenge weak ideas.

Improve features where appropriate.

Optimize for long-term maintainability and scalability.

Vision

The application is an AI-powered Islamic Companion Platform.

AI is only one part of the product.

The application should become one of the most complete Islamic apps available.

The product must feel premium, modern, beautiful, fast, and production-ready.

The experience should exceed the current application in usability, quality, performance, and features.

Whenever you believe an additional feature improves the product without making it unnecessarily complex, propose it or implement it.

Platforms

The platform consists of:

- iOS Application (SwiftUI, iOS 17+)
- Android Application (Jetpack Compose, Kotlin)
- Admin Dashboard (Web)
- Backend API

The Admin Dashboard is completely separate from the mobile applications.

Architecture

Use a production-grade architecture.

Clean Architecture

MVVM-C

Coordinator Pattern

Factory Dependency Injection

Feature-first modularization

Core

UICore

Design System

Networking

Repository Pattern

Use Cases

System Design

The architecture must be easy to extend for years.

Backend

Backend:

Supabase

Firebase only for:

- Push Notifications
- Analytics
- Crash Reporting

The mobile applications must NEVER communicate directly with Supabase.

Architecture:

Presentation

↓

Use Cases

↓

Repositories

↓

APIClient

↓

APIEndpoints

↓

Backend API

↓

Supabase

The backend owns all business logic.

The mobile applications should never know whether the backend is implemented using Supabase Edge Functions or another backend technology.

Use versioned REST APIs.

Everything should remain replaceable.

Main Modules

Authentication

Home

Prayer

Qibla

Azkar

Dua

Tasbeeh

Gamification

Leaderboard

Search History

Notifications

Widgets

Settings

AI Search

AI Hadith Extraction

AI Islamic Questions

Admin Dashboard

Configuration

Analytics

Home

The Home screen contains three AI features:

- ابحث عن فتوى
- استخراج الأحاديث
- سؤال ديني عام

Do not implement these first.

First Development Priority

The first implementation priority is the "مزايا" section.

Complete every feature inside this section before beginning the AI search features.

Modules include:

- Prayer Times
- Qibla
- Azkar
- Dua
- Digital Tasbeeh
- Search History
- Leaderboard

Design these modules as complete production features.

Gamification

The application includes a complete streak system.

Requirements:

- Multiple streak categories.
- One overall streak.
- Each streak represented using the application's own branding instead of a fire emoji.
- Users may optionally publish a display name.
- Leaderboards:
 - Global
 - Country
- Admin can create challenges:
 - For one streak
 - Multiple streaks
 - Overall streak
- Reward system

- Achievement system
- Daily challenges
- Weekly challenges

The gamification system should be engaging while respecting the application's Islamic identity.

Notifications

Implement a powerful notification engine.

Examples include:

- Third Night reminder
- Before Adhan reminder
- Before Iqamah reminder
- Morning Azkar
- Evening Azkar
- Daily reminders
- Challenge reminders
- Streak reminders

Users must be able to enable or disable every notification independently.

Users can configure reminder offsets.

Each notification option should contain a help icon explaining its purpose.

Widgets

Build widgets from the beginning.

Support widgets for:

- Prayer Times
- Next Prayer
- Prayer Countdown
- Azkar
- Tasbeeh Counter
- Daily Challenge
- Streak
- Hijri Date

Use modern WidgetKit (iOS) and Android App Widgets / Glance.

Admin Dashboard

The dashboard manages:

Users

Prayer Settings

Challenges

Leaderboards

Notifications

Azkar

Duas

Hadith Collections

Fatwa Sources

AI Configuration

Knowledge Base

Announcements

Feature Flags

Application Configuration

Content Management

Analytics

The dashboard is the source of truth.

AI

The AI must always prioritize trusted Islamic sources.

Every answer should include references whenever possible.

Design the AI layer so providers can be replaced later.

Supported providers should be abstracted behind interfaces.

Design

The application designs already exist inside the repository.

Study them carefully.

However:

Do not copy them blindly.

Improve them.

Make interactions smoother.

Improve navigation.

Improve hierarchy.

Improve accessibility.

Improve spacing.

Improve typography.

Improve animations.

Improve onboarding.

Improve empty states.

Improve loading states.

Improve micro-interactions.

If you identify UX improvements that make the product stronger, implement them.

The objective is to create an application that is visually and functionally superior to the current design.

Documentation

Everything must be documented.

Maintain:

Architecture documentation

API documentation

ADRs

Roadmap

Changelog

Developer Guides

Setup Guides

Implementation Notes

Future Improvement Notes

Repository Documentation

The repository should be understandable by another senior engineer without additional explanation.

Development Process

Before writing production code:

- Analyze the project.
- Produce a complete implementation roadmap.
- Review the architecture.
- Challenge important design decisions.
- Identify potential improvements.

Then begin implementation.

Do not ask me what to implement next.

Plan your own milestones.

Keep documentation synchronized.

Refactor when necessary.

Avoid technical debt.

Continue implementation until the platform is complete.

Only stop if:

- A major product decision requires approval.
- An architectural decision has multiple valid solutions requiring business input.
- External credentials or assets are required.

Otherwise continue building autonomously.

Build a product that is significantly better than the current application rather than merely recreating it.